

Task: Graph Multi-Objective Routing (GraphMOP)

Overview

You are given an undirected network graph in which a few nodes host **servers** and the rest are **terminal devices** that continuously generate computational tasks. Your job is to assign each task source to a server and choose a path through the graph that delivers the task's traffic to that server.

The challenge is to balance two competing objectives across the whole system: **end-to-end latency** and **total energy consumption**. The two are not independent — sending every task to the closest server overloads it (latency explodes), while spreading tasks over distant servers raises both transmission energy and hop counts.

Input

Parameter	Description
<code>num_nodes</code>	Total number of nodes in the graph, indexed $0, 1, \dots, \text{num_nodes} - 1$.
<code>edges</code>	List of undirected edges. Each edge is a tuple (u, v, ℓ, e, c) where ℓ is the base latency , e is the energy cost per unit traffic , and c is the link capacity .
<code>server_nodes</code>	List of node indices that host servers. Tasks must terminate at one of these nodes.
<code>task_sources</code>	List of node indices that generate tasks. Every source must be assigned a server and a path.
<code>task_info</code>	Dict mapping each source to a tuple (λ, s, w) , where λ is the arrival rate , s is the payload size , and w is the compute demand per task.
<code>server_info</code>	Dict mapping each server to a tuple $(\mu, P_{\text{idle}}, P_{\text{dyn}})$, where μ is the server's service rate , P_{idle} is the idle power , and P_{dyn} is the dynamic power coefficient.

Output

A dict keyed by source node, where each value specifies the chosen server and the routing path:

```

1 {
2     source_node: {
3         "server": server_node,
4         "path": [source_node, ..., server_node],
5     },
6     ...
7 }

```

The plan must satisfy these constraints — violating any of them invalidates the entire solution:

- Every node in `task_sources` must appear as a key.
- `server` must be a member of `server_nodes`.
- `path` must start at `source_node` and end at `server`.
- Every consecutive pair in `path` must be a real edge in the graph.
- `path` must be a **simple path** (no repeated nodes, no cycles).
- For every server, the total assigned compute load must stay strictly below its service rate μ .

Scoring

Your submission is evaluated by a final **Score**, defined as:

$$\text{Score} = -\overline{F}_{\text{lat}} \times \overline{F}_{\text{eng}} \times (1 + 0.05 \cdot \overline{H} + 0.03 \cdot A)$$

A higher Score is better. The internal grader actually minimizes the positive quantity $\overline{F}_{\text{lat}} \times \overline{F}_{\text{eng}} \times (1 + 0.05 \cdot \overline{H} + 0.03 \cdot A)$ across several test graphs and reports the negation as `combined_score`.

The two primary factors enter **multiplicatively**, so a catastrophic value on either latency or energy ruins the overall Score regardless of the other factor. The two additive terms — average hop count $\overline{H}$ and number of active servers A — act as gentle regularizers that discourage pointlessly long paths and pointlessly many active servers.

Factor Definitions

Let the plan assign source i to server $\sigma(i)$ along path $\pi_i = (i = v_0, v_1, \dots, v_{h_i} = \sigma(i))$. Let $\Lambda = \sum_i \lambda_i$ denote the total arrival rate over all sources.

For each edge (u, v) , the **edge load** is the sum of traffic of all tasks whose path uses that edge:

$$L(u, v) = \sum_{i: (u, v) \in \pi_i} \lambda_i \cdot s_i$$

For each server k , the **server load** is the sum of compute load of all tasks routed there:

$$L_{\text{srv}}(k) = \sum_{i: \sigma(i)=k} \lambda_i \cdot w_i$$

F_lat — End-to-End Latency

The latency of a single task is the sum of its **transmission latency** along the path and the **queueing delay** at its assigned server. The transmission latency along an edge grows linearly with that edge's load ratio:

$$T_i = \sum_{(u,v) \in \pi_i} \ell_{uv} \cdot \left(1 + 0.55 \cdot \frac{L(u,v)}{c_{uv}}\right)$$

The queueing delay at server k follows a simple M/M/1-style formula and explodes as the load approaches capacity:

$$Q_k = \frac{1}{\mu_k - L_{\text{srv}}(k)}, \quad \text{requires } L_{\text{srv}}(k) < \mu_k$$

The reported average latency is the **arrival-rate-weighted mean** across all sources:

$$\overline{F}_{\text{lat}} = \frac{1}{\Lambda} \sum_i \lambda_i \cdot (T_i + Q_{\sigma(i)})$$

F_eng — Total Energy

The energy budget has two components. **Routing energy** is the energy spent moving traffic across edges:

$$E_{\text{route}} = \sum_i \sum_{(u,v) \in \pi_i} \lambda_i \cdot s_i \cdot e_{uv}$$

Compute energy is paid by every server that has any load on it. An idle (turned-off) server pays zero. An active server pays its idle power plus a sub-linear dynamic-power term:

$$E_{\text{compute}} = \sum_{k : L_{\text{srv}}(k) > 0} \left(P_{\text{idle}}^{(k)} + P_{\text{dyn}}^{(k)} \cdot u_k^{1.35} \right), \quad u_k = \frac{L_{\text{srv}}(k)}{\mu_k}$$

The reported average energy is normalized by total arrival rate:

$$\overline{F}_{\text{eng}} = \frac{E_{\text{route}} + E_{\text{compute}}}{\Lambda}$$

Regularizers

The average hop count $\overline{H}$ is the arrival-rate-weighted mean path length:

$$\overline{H} = \frac{1}{\Lambda} \sum_i \lambda_i \cdot h_i$$

The active-server count A is simply the number of distinct servers with non-zero load:

$$A = |\{k : L_{\text{srv}}(k) > 0\}|$$

Key Observations

- **Latency and energy are coupled through server load.** Shorter paths reduce both transmission latency and routing energy, but they tend to concentrate tasks on a few nearby servers, which raises queueing delay (the $1/(\mu - L)$ term grows without bound as load approaches the service rate).
- **The multiplicative structure is unforgiving.** A plan that looks reasonable on average but pushes a single server near its capacity will see latency explode and drag the

whole Score down.

- **Turning a server off saves energy, but only if you can fit the offloaded tasks elsewhere.** Each active server incurs a fixed idle-power cost; consolidating onto fewer servers saves energy *if* the remaining servers can absorb the load comfortably.
- **A good plan jointly chooses three things:** which subset of servers to keep active, how to assign each source to a server, and which path to use to reach it. Greedy nearest-server assignment is a poor baseline because it ignores both load-balancing and energy consolidation.